

Lab 3: Component Modelling & Architectural Pattern Selection

Prasanna Chidambar Marihalkar | PES1UG24CS684 | Hospital Bed & ICU Allocation Dashboard

Architecture Selection

We chose **Client-Server Architecture** for the **Hospital Bed & ICU Allocation Dashboard** System.

Architectural choice: Client-Server. Triage nurses and hospital administrators use a thin Dashboard UI (client tier), while allocation, availability, bed-status, access-control and data components run together on one hospital server (server tier), as shown in the component diagram.

Style	Strength for this system	Weakness for this system	Verdict
Layered	Clean UI / logic / data separation; easy to understand.	Does not by itself address many workstations sharing one live bed state; layer overhead.	Medium
Microservices	Independent scaling and fault isolation.	Heavy operations effort; bed state split across services risks inconsistency for a single hospital.	Low
Client-Server	One central, authoritative copy of bed / ICU data with thin clients.	Central server is a single point of failure and a bottleneck.	Chosen

Two Reasons

1. **One authoritative view of bed and ICU state.** Several ER workstations and the administrator must see the same live occupancy. Keeping the database and the Bed Status Manager on one server means two nurses cannot be offered the same last ICU bed, and consistency is easy to guarantee. Splitting bed state across microservices would make this harder.
2. **Allocation rules live in one place.** Triage-score ICU prioritisation and inter-facility transfer rules sit in the Bed Allocation Manager. A rule change is deployed once on the server and every ward workstation gets it with no client update, which keeps deployment simple for a single hospital.

Security Advantage

Access is enforced centrally at the server. Every request from the Dashboard UI passes the Auth & Audit Service (IAuthentication, IAccessCheck), which checks the session token and the role (Triage Nurse or Administrator) before the Bed Allocation Manager touches any data, and logs each allocation for audit. The Hospital Bed Database is reachable only from the server tier, never directly from a client, and client traffic uses HTTPS/TLS. Patient-related bed data is therefore protected in one controlled backend instead of on every workstation.

Performance Benefit

Clients stay lightweight because they only render and send requests. Availability queries and bed-status writes (IBedData, IStatusData) run on the server next to the database, so one dashboard refresh costs the client a single call to IAllocationAPI instead of several database round trips. The server can also cache frequently read occupancy counts and index the bed table, so repeated dashboard refreshes do not repeat the same work.

Trade-off

The central server is a single point of failure and a possible bottleneck. For a hospital this is reduced by running a standby server and regular database backups. The consistency and central-control benefits still outweigh this risk for a single-hospital dashboard.